LcsHelper v4 — Algorithm Overview

ULTRASAT Low Cadence Survey scheduler (Weizmann Institute of Science)

Overview

LcsHelper v4 is a MATLAB class that produces an observation schedule for ULTRASAT's Low Cadence Survey (LCS). ULTRASAT is the Israeli/WIS UV space telescope, scheduled to launch in 2027–2028; the LCS is one of its two main observing modes (the other being the High Cadence Survey).

The class produces a schedule for up to ~84 sky fields (selected from ~200 on the all-sky non-overlapping grid) over ~360 days, respecting hard constraints from the spacecraft's exclusion angles (Sun ≥ 70°, Moon ≥ 34°, Earth ≥ 56°, plus ULTRASAT's solar-panel pointing limit). The output is a tiered observation plan grouped into four field categories (A, B, C, D) with different cadences and dwell durations, plus a per-day, per-slot daily timetable.

1. Field categories at a glance

Set	N fields	Dwell	Cadence	Selection criterion
A	48	45 d	1 d	$A_U \le 1$, 45 d ≤ visibility < 135 d (~42 fields), plus the worst-extinction long-window fields to reach 48
B	16	45 d + 90 d	1 d / 4 d	$A_U \le 1$, longest window ≥ 135 d, best extinction
C	16	135 d	4 d	$A_U \le 1$, longest window ≥ 135 d, next-best extinction
D	up to 4	45 d	1 d	$A_U > 1$, hard-coded WG5 ranking

A SetB field is observed continuously across 3 consecutive 45-day windows: one window at 1-day cadence (its B_45 "high-cadence" sub-block) and two at 4-day cadence (its two B_90 "low-cadence" sub-blocks). A SetC field is observed across 3 consecutive 45-day windows entirely at 4-day cadence. SetA and SetD fields each occupy a single 45-day window at 1-day cadence.

1.1 A tight field budget

A consequence of the selection criteria is that the field pool has very little slack. For typical survey start dates only ~80–81 fields meet the $A_U \le 1$ good-extinction criterion at all, and the scheduler needs exactly 48 + 16 + 16 = 80 of them for Sets A, B, and C combined. The partition into Sets A/B/C is almost forced rather than chosen — the ~6 long-window fields that fall into Set A are simply whichever long-window good-extinction fields didn't fit into Sets B (best 16) or C (next 16).

When the categorisation's greedy choices make a particular slot unfillable, the scheduler has limited room to retreat. It absorbs such failures via two mechanisms: (a) the variant-based SetB/SetC matching (Section 3) gives the scheduler 4 different block layouts to try per attempt, and (b) the field-shuffle mechanism (Section 9) swaps problematic fields between sets using the small `Long_leftover_fields` pool (~11 fields).

2. Astronomical context and design choices

2.1 Extinction is the only sky-quality filter

UV galactic extinction (A_U) is the dominant noise source for ULTRASAT's transient hunt, so $A_U \leq 1$ mag is a hard gate for Sets A/B/C. Set D exists because WG5 (the science working group) wants four high-extinction targets observed regardless of this gate; SetD candidates are looked up by ID from the all-sky table.

2.2 Cadence triplet matches transient timescales

The 45 d / 90 d / 135 d cadence triplet matches Type Ia and core-collapse supernova lightcurve timescales: 45 d at 1 d cadence catches rise + peak, 90 d at 4 d cadence catches the tail, 135 d covers slow transients. Set B fields get all three (one B_45 + two B_90 sub-blocks); Set C is tail-only (entirely 4 d cadence); Set A is the high-cadence-only fast-transient pool.

2.3 Two visibility modes

`Whole_daily_window = true` (`vis2d_day_field_ALL`) is the conservative mode — a field counts as "visible" on day d only if it stays inside all four exclusion cones for the entire 3-hour LCS observing window. The default `false` (`vis2d_day_field_ANY`) accepts a field if even one 15-minute slot is clean. The downstream cadence and dwell-window logic doesn't change, but the field pool grows considerably under the lenient mode.

3. Pipeline overview

The scheduler runs in two phases. **Preparation** computes visibility tables and produces the initial field categorisation. **Scheduling** places A/B/C with retry, then SetD, then builds the per-day schedule matrix.

Step	Method	Purpose
1	calc_vis_matrix	Per-day visibility per field from Sun/Moon/Earth exclusion
2	calc_cont_vis_windows_v2	Continuous-visibility runs per field, with 1d-gap variant
3	categorizeFields_v4	Initial extinction-only split into Sets A/B/C + Long_leftover
4	categorize_then_schedule	Orchestrator: SetA, then 4 variants of SetB/SetC, then SetD, then daily schedule

Steps 1-3 run inside `prepTablesBeforeSchedule`. Step 4 runs inside `categorize_then_schedule` and internally invokes `schedule_SetA_v4`, `schedule_SetC_v4`, `schedule_SetB_v4`, `schedule_SetD_v4`, and `calcDailySchedule`.

On a typical StartDate sweep, the full A/B/C/D pipeline succeeds in ~74% of cases. The scheduler is built on extinction-only field categorisation, bipartite matching (`matchpairs`) for slot assignment, a catalogue of 4 OPTIMAL window-occupancy variants for SetB and SetC (Section 4), and a per-rank case-based placement for SetD.

4. The 4 OPTIMAL SetB/SetC variants

SetB and SetC share the same 8-window time grid as SetA, but their fields cover 3 contiguous windows each. A "block" B_k denotes the triple of windows (W_k, W_{k+1}, W_{k+2}) for k = 1..6. Each SetC and SetB field is assigned to one such block.

A Set C field contributes 1/4 to each of its 3 windows (4-day cadence throughout). A Set B field contributes 1 to one designated "1"-window (its B_45 high-cadence sub-block) and 1/4 to each of the other two (its two B_90 low-cadence sub-blocks).

With 16 SetB + 16 SetC fields and an 8-window grid, the total weighted occupancy is 16×3×1/4 + 16×3/2 = 36, averaging 4.5 per window. An OPTIMAL configuration achieves maximum window-occupancy of 5, with exactly four windows at 5 and four at 4 (no window over-budget).

Each variant is fully specified by:

- `C_blocks(k)` — how many SetC fields use block B_k (sum = 16).
- `B_blocks(k)` — how many SetB fields use block B_k (sum = 16).
- `ones_at(k, j)` — among the B_k SetB fields, how many have their "1"-window at W_j (where j is one of k, k+1, k+2).

Variant	C_blocks (B_1..B_6)	B_blocks (B_1..B_6)	Occupancy (W_1..W_8)
Optimal 1	(4, 4, 0, 0, 4, 4)	(4, 2, 2, 2, 2, 4)	(5, 5, 4, 4, 4, 4, 5, 5)
Optimal 2	(0, 4, 4, 4, 4, 0)	(4, 3, 1, 1, 3, 4)	(4, 5, 4, 5, 5, 4, 5, 4)
Optimal 3	(4, 4, 0, 4, 4, 0)	(4, 2, 2, 2, 2, 4)	(5, 5, 4, 5, 5, 4, 4, 4)
Optimal 4	(0, 4, 4, 0, 4, 4)	(4, 2, 2, 2, 2, 4)	(4, 4, 4, 5, 5, 4, 5, 5)

The four variants are stored as a Constant property `Obj.Variants`, a `1×4` struct array with fields `name`, `C_blocks`, `B_blocks`, `ones_at` (6×8 matrix).

5. Categorisation (`categorizeFields_v4`)

Step 3 partitions the field pool using only extinction and visibility-window-length thresholds. No anchor awareness, no scheduling-feasibility look-ahead.

5.1 Filter cascade

Filter	Condition
Low_ext_fields	$A_U \leq$ max_ext (=1.0) AND max_window ≥ Min_window (=45 d)
Long_low_ext	Low_ext_fields AND max_window ≥ Max_window_cut (=135 d)
Short_fields	Low_ext_fields \ Long_low_ext

If `Low_ext_fields` contains too few fields for A+B+C+1 = 81 candidates, the filter is relaxed to use the 1-day-gap-allowed visibility tables. Same fallback applies if `Long_low_ext` has fewer than 32 fields. The `use1gap` flag is recorded per field so the right visibility table is used downstream.

5.2 Set assignment

After sorting `Long_low_ext` by extinction ascending, the cleanest-extinction fields get assigned first:

Set	Source	Count
SetB	Long, lowest-extinction 16	16
SetC	Long, next-lowest-extinction 16	16
SetA	Short_fields + remaining Long	~48
Long_leftover	Same as "remaining Long", kept as a candidate pool for shuffling	~11

Categorisation runs ONCE at the start of `categorize_then_schedule`. The orchestrator never resets it — the extinction ranking is preserved across all variant attempts and shuffle iterations. Only `shuffle_on_failure` mutates the field-table assignments.

6. Time-grid definitions

All scheduling uses a fixed 8-window time grid anchored at `First_day`:

```
Nominal_windows.start = First_day + Min_window * (0:7)
Nominal_windows.end   = Nominal_windows.start + Min_window - 1
Full_windows          = Nominal_windows
```

With `First_day = 1` and `Min_window = 45`, the eight Full_windows are [1,45], [46,90], [91,135], [136,180], [181,225], [226,270], [271,315], [316,360]. A SetB or SetC block B_k covers windows W_k, W_{k+1}, W_{k+2} for 135 days starting at `Full_windows.start(k)`.

SetA grid divides these 8 windows into 6 SetA groups of 8 slots each:

```
SetA slot start = anchor_g + (s-1) * Min_window
where g = group (1..6), s = slot (1..8),
and anchor_g = First_day + (g-1) * 8 * Min_window  for the un-shifted case
```

7. The orchestrator: `categorize_then_schedule`

The orchestrator drives a retry loop where each attempt tries all 4 variants. The loop structure is: variants INSIDE each shuffle attempt. This preserves the categorisation's extinction ranking across as many variant attempts as possible before mutating it via shuffle.

7.1 Control flow

```
Args (defaults):
  MaxRetries    = 10
  RunSetD       = true
  VariantOrder  = [1 2 3 4]
  SkipVariants  = []

Obj.Variant_used = 0
categorizeFields_v4()                   # ONCE; ranking preserved

for attempt = 1..MaxRetries:
    Obj.Schedule = empty
    [okA, unp_A] = schedule_SetA_v4()   # variant-independent; ONCE per attempt
    if not okA:
        shuffle_on_failure(unp_A, [], [])
        continue (or break if no shuffle)
    snapshot Obj.Schedule and Obj.SetA_shifted_group

    for v in VariantOrder \ SkipVariants:
        restore Obj.Schedule from snapshot     # SetA-only state
        [okC, unp_C] = schedule_SetC_v4(v)
        [okB, unp_B] = schedule_SetB_v4(v)
        store unp_C, unp_B per variant
        if okC and okB:
            Obj.Variant_used = v
            success; break both loops

    if no variant succeeded:
        [unp_C*, unp_B*] = local_pick_most_common_unplaced(\
                              unp_C_per_variant, unp_B_per_variant)
        shuffle_on_failure([], unp_C*, unp_B*)
        if no shuffle: break

if success:
    if RunSetD: schedule_SetD_v4()
    try calcDailySchedule() catch warn
```

7.2 Why this loop structure works

On each shuffle attempt, the orchestrator gets 4 chances (one per variant) to find a feasible SetB/SetC placement. If variant 1 can't fit field X into a SetC slot but variant 3 can, success is reached without needing to shuffle X away. This is efficient because:

- Categorisation is preserved across all 4 variants (no re-shuffle penalty when switching variants).
- SetA runs only once per attempt; the post-SetA state is snapshotted and restored between variants.
- 4 placements per shuffle attempt span a wide geometric range, with each variant putting SetC and SetB fields in different blocks.
- Each optimal variant leaves exactly 4 inds with 1 free slot — perfect for SetD's 4 fields with no pre-clean needed.

8. schedule_SetA_v4

Places all 48 SetA fields by solving a bipartite max-cardinality matching. Two phases:

Phase 1. All 6 SetA groups anchored at `First_day`. Build a 48-slot bipartite graph (slots vs SetA fields) and solve with `matchpairs`. Edge cost is 0 if the field fits the slot, BIG (=1e6) otherwise. If matchpairs finds a perfect matching, done.

Phase 2. If Phase 1 leaves any slot unfilled, try shifting exactly one group by ±1..30 days. For each (`g_shift`, sh), rebuild slot starts with group `g_shift` at the new anchor and rerun matchpairs. Take the first configuration that achieves full 48-field placement. If only partial improvements are found, keep the best partial result.

When Phase 2 commits any configuration involving a shifted group, the shifted group index is recorded in `Obj.SetA_shifted_group`. This is consumed later by the SetD pre-clean and Case B logic (Section 11): SetA fields in the shifted group are ineligible to be moved because their visibility is tied to the shifted anchor, not the standard time grid.

9. Variant-based SetC and SetB scheduling

9.1 `schedule_SetC_v4(variant_idx)`

Reads `Obj.Variants(variant_idx).C_blocks`. Builds a slot list of length 16, where each slot is tagged with its block index k (each k repeated `C_blocks(k)` times).

Cost matrix (16 SetC fields × 16 slots): cost 0 if the field has 135-day continuous visibility starting at `Full_windows.start(k)`, BIG (=1e6) otherwise. `matchpairs` returns the assignment; success requires all 16 slots matched. On failure, the unplaced field IDs are returned.

Schedule row construction (SetC convention):

- `category = "C"`
- `group = 10 + block_index` — so groups 11..16 (one per block)
- `ind = 1..C_blocks(k)` — within-block counter
- `start = Full_windows.start(k)`; `end = start + 3*Min_window - 1`

9.2 `schedule_SetB_v4(variant_idx)`

Reads `B_blocks` AND `ones_at`. Builds a slot list of length 16, where each slot is tagged with (block k, "1"-window j). Cost matrix is (16 SetB fields × 16 slots): cost 0 if the field has 135-day visibility starting at `Full_windows.start(k)`, BIG otherwise — the "1"-window j doesn't affect feasibility (it's a sub-block label only).

For each matched (field, slot), three Schedule rows are emitted:

- **1 B_45 row** at the "1"-window W_j: `category="B_45", group=100+j, ind=within-group counter, start/end=Full_windows for` W_j
- **2 B_90 rows** at the other two windows of block k: `category="B_90", group=200+w, ind=within-group counter, start/end=Full_windows for` W_w

`ind` is a counter shared across all SetB fields contributing to the same group (e.g., if 4 B_45 fields are at W_1, they get ind = 1, 2, 3, 4). This convention is what makes `calcDailySchedule`'s `mod(ind, 4)` cadence math spread fields across the four cadence offsets correctly.

10. Shuffle on failure

When all 4 variants fail in a given attempt, the orchestrator aggregates the unplaced fields across all variants and picks ONE field to shuffle. The principle: shuffle the most-blocked field, since it's likely the real obstacle.

10.1 `local_pick_most_common_unplaced`

Inputs: two cell arrays of column vectors, one per variant, containing unplaced SetC (resp. SetB) fields from that variant.

Algorithm:

- Aggregate (concatenate) all unplaced SetC lists across variants; compute the field with the highest count (call it f_C, count n_C).
- Same for SetB: f_B, n_B.
- If `n_C > n_B`: return (f_C, []) — shuffle SetC.

- If `n_B > n_C`: return ([], f_B) — shuffle SetB.
- On tie (including both empty): return both. `shuffle_on_failure` dispatches SetB first by its internal priority.

10.2 `shuffle_on_failure` dispatch

`shuffle_on_failure` applies one swap per call. It dispatches based on which scheduling step failed:

Failure type	Primary action	Fallback
SetB failed	Swap unplaced SetB field with lowest-extinction Long_leftover field	Swap with highest-extinction SetC field
SetC failed	Swap unplaced SetC field with lowest-extinction Long_leftover field	Swap with highest-extinction SetB field
SetA failed	Move a Long_leftover field from SetA into SetC	(only one path tried; if it fails the orchestrator gives up)

11. schedule_SetD_v4

SetD places up to 4 high-extinction fields, chosen in priority order from a fixed WG5-supplied ranking `[79 12 48 28 16 88 55 32 213 26]`. Each placement uses one ind of `Full_windows` and a single 45-day @ 1-day-cadence observation.

The SetD pipeline has three logical phases: **slot accounting**, **pre-clean** (to clear any over-budget inds left by A/B/C), and a **per-rank placement loop**. The invariant is that every A/B/C field in `Obj.Schedule` remains scheduled throughout (possibly relocated to a different ind, but never removed).

11.1 Slot accounting

`local_compute_slot_occupancy` computes the per-ind "filled slot count" from the existing A/B/C schedule. The accounting is cadence-aware: SetB_90 and SetC fields share daily slots in groups of 4 (one per cadence offset mod 4).

```
filled(k) = nA(k) + nB45(k) + (nB90(k) + nC(k)) / 4

where:
  nA(k)   = SetA  rows with ind == k
  nB45(k) = B_45  rows with group == 100 + k  (Full_windows ind
            recovered from group, since B_45.ind is a counter)
  nB90(k) = B_90  rows with group == 200 + k  (same convention)
  nC(k)   = SetC  rows whose super-window covers ind k
          (each SetC field covers 3 consecutive inds)
```

For the 4 optimal variants, slot accounting yields `inds_2move` = [] (no over-budget inds) and `inds_open` with 4 entries:

Variant	inds_open
Optimal 1	[3, 4, 5, 6]
Optimal 2	[1, 3, 6, 8]
Optimal 3	[3, 6, 7, 8]
Optimal 4	[1, 2, 7, 8]

11.2 Pre-clean

Before placing any SetD field, `clean_inds_before_setD` ensures `inds_2move` is empty by moving SetA fields from over-budget inds to inds in `inds_open`. Done globally via `matchpairs`. For all 4 optimal variants this is a no-op (no over-budget inds), but the step is essential to maintain the invariant in any future schedule that lands with `inds_2move` non-empty.

11.3 Per-rank placement loop (Case A / B / C)

Iterate the WG5 ranking with a rank pointer; for each SetD slot, advance through ranks until a placement commits or the ranking exhausts. For each candidate:

- Compute feasible inds for this candidate (those where it has 45-day visibility starting at `Full_windows.start(k)`).
- **Case A — direct placement.** If any feasible ind is in `inds_open`, place the SetD field there and consume one entry from `inds_open`.

- **Case B — bump a SetA field.** Otherwise, for each feasible ind `k_setD` NOT in `inds_open`, search SetA rows at `k_setD` (non-shifted group) for a field that has visibility at some `k_open` in `inds_open`. Pick the candidate with the FEWEST alternative inds (most-constrained), move it to `k_open`, place SetD at `k_setD`.
- **Case C — skip.** If neither A nor B finds a placement, skip this rank and advance the pointer.

12. Daily schedule (`calcDailySchedule`)

The final step of `categorize_then_schedule` builds `Obj.Daily_schedule`: a matrix of size `(Last_day - First_day + 1) x Daily_LCS_slots` (typically 360 × 11) where entry `(d, s)` is the field ID observed in slot s on day d (NaN if no observation).

For each row in `Obj.Schedule`, the function fills the appropriate days according to the cadence rule for that category:

Category	Cadence	Daily placement rule
A	1 day	Every day in [start, end]
B_45	1 day	Every day in [start, end]
B_90	4 days	Days where mod((d - start + 1), 4) == mod(ind, 4)
C	4 days	Days where mod((d - start + 1), 4) == mod(ind, 4)
D	1 day	Every day in [start, end]

After populating, the function performs a within-day slot reordering: for each day, fields that are not visible across ALL slots get relocated to a slot where they are visible (per `vis3d_slot_day_field`). The function is wrapped in a `try/catch` by the orchestrator so any unexpected error issues a warning but does not invalidate `Obj.Schedule`.

13. Plotting tools

Two inspection helpers are included for visualisation. They are not called by the core pipeline.

Method	Purpose
plotSchedule(Args)	Timeline of all scheduled fields across categories A/B/C/D with category band markers
plotCatB(Args)	Per-field timeline for the 16 SetB fields (solid line = W45, dashed = W90 blocks)

13.1 SetC plotting

Each SetC group covers a 3-window block, and multiple SetC groups can overlap in time (e.g., block B_1 covers days [1,135] and block B_2 covers [46,180]). To visualise this, `plotSchedule`:

- Performs a greedy interval coloring on the SetC groups: groups whose time spans overlap are assigned different lines; non-overlapping groups can share a line.
- Lines are placed at fixed y-positions: y = 17, 18, 19, ... (1-unit spacing).
- Within each line, fields are offset tightly by `(ind-1)/8` — partial-line separation matching the B_90 visual style.

13.2 Other categories

SetA rows are plotted at `y = group`; the original 6 groups map to y = 1..6. Fields moved by the SetD pre-clean or Case B receive groups ≥ 7. Group 7 still falls in the SetA band; groups 8+ may visually drift into the SetB band — this is cosmetic only. SetB_45 and SetB_90 are plotted within bands y in [9,16]. SetD is plotted within the band y in [21,24]. Category bands are marked with horizontal lines at y = 8 (A/B), 16 (B/C), 20 (C/D), 25 (D/top).

14. Diagnostic output (`Verbose` flag)

All diagnostic `fprintf` calls are gated on `Obj.Verbose`, which defaults to `false`. A typical StartDate sweep produces no output unless a `warning(...)` is emitted (e.g. the slot-occupancy divisibility check or the "no feasible schedule" warning).

To turn diagnostics on, pass `Verbose=true` at construction:

```
LCS = ultrasat.planner.LcsHelper_v4( ...
    'StartDate', '2029-02-01', ...
    'AllSkyTable', tbl, ...
    'Verbose', true);
LCS.prepTablesBeforeSchedule;
LCS.categorize_then_schedule;
```

When verbose, the orchestrator prints per-attempt and per-variant progress (showing the variant name and which set failed), shuffle decisions with the field IDs involved, SetD Case A/B/C decisions, and pre-clean SetA moves.

15. Data structures

Property	Type	Role
vis3d_slot_day_field	logical [slots x days x fields]	Raw visibility per 15-min observing slot
vis_day_field	logical [days x fields]	Per-day visibility
Cont_visibilty_per_field	double [days x fields]	Length of visibility run starting at each (day, field)
Cont_visibilty_per_field_1d gap	double [days x fields]	Same but treating isolated 1-day gaps as still visible
Longest_window_per_field	double [1 x fields]	Maximum entry per column (longest run length)
SetA_fields, SetB_fields, SetC_fields	table	Field assignments; Field/Av_ext/use1gap/max_window/scheudled
Long_leftover_fields	table	Long-visibility fields not in B or C (initially ~11)
SetD_ranked_fields	table	WG5 ranking with placement state
Variants	struct array (Constant, 1x4)	Catalogue of Optimal 1..4: name, C_blocks, B_blocks, ones_at
Variant_used	double (0..4)	Which variant succeeded (0 = none)
inds_open	array (with repetition)	After SetD: inds with free slots; ind k repeated per open count
inds_2move	array (with repetition)	After SetD: should be empty (cleared by pre-clean)
Nominal_windows, Full_windows	table (start, end)	Time grid: First_day + 45*(0:7)
SetA_shifted_group	double (0..6)	Group shifted by SetA Phase 2 (0 = none); SetD moves exclude this group
Schedule	table	Final output: category/group/ind/start/end/Field rows
Daily_schedule	double [days x slots]	Per-day per-slot matrix; entry = field ID or NaN

16. Helper functions (outside classdef)

Helper	Used by	Purpose
local_make_field_table	categorizeFields_v4	Build a standard 5-column field table from a list of field IDs
local_match_setA	schedule_SetA_v4	Bipartite max-cardinality matching for SetA slots vs fields
local_pick_most_common_unplaced	categorize_then_schedule	Aggregate per-variant unplaced lists; return single most-common field per set
local_unplaced_winner	local_pick_most_common_unplaced	Count field occurrences across a cell array of lists; return the winner

local_compute_slot_occupancy	clean_inds_before_setD	Compute inds_open / inds_2move from Schedule (cadence-aware, /4 check)
local_apply_setA_moves	clean_inds_before_setD	Apply (Field, from_ind, to_ind) moves to SetA rows; assigns new group
local_assign_moved_setA_group	local_apply_setA_moves / Case B	Lowest group ≥ 7 at target ind that is not already taken (7, 8, 9, ...)
local_commit_setD	schedule_SetD_v4	Append a SetD row to Schedule (category="D", group=300+slot)
consecutive_trues_cols	calc_cont_vis_windows_v2	Count consecutive-true run length per column of a logical matrix
fill_isolated_gaps	calc_cont_vis_windows_v2	Fill single-false gaps in a logical matrix (1-0-1 → 1-1-1)

Companion file: LcsHelper_v4.m (~2000 lines).